PYTHON LISTS

➤ WHAT IS LIST ?!

In Python, list is one of the built-in collection/container data types, Lists are used to store multiple data at once.
Lists are created in Python, using the square brackets [ ]. Each element in the list is separated by comma. In the list, we can hold any object type like string, boolean, float or even list inside the list.

Example1:

```
names_list = ['Steve', 'Sundar', 'Rahul', 'Sandeep']
print (names_list)
```

Output:

```
['Steve', 'Sundar', 'Rahul', 'Sandeep']
```

Example 2:

```
emp_details = ['Rahul', 'Kumar', 'Sales', 380]
print (emp_details)
```

Output:

```
['Rahul', 'Kumar', 'Sales', 380]
```

➤ type() function :⇒

The type() function will return the 'list' keyword.

Example:

```
print (type(emp_details))
```

Output:

```
<class 'list'>
```

➤ LIST INDEXING

Each item is assigned the index position starting with 0. We can access each item by passing the index position in the square brackets.

Example:

```
names =['Steve', 'Sundar', 'Rahul', 'Sandeep', 'Vivek']
print (names[2])
```

Output:

```
Rahul
```

➤ LIST SLICING

The slicing operation can extract a list of two or more than two items from the list. While performing slicing, we can pass three optional parameters in the square brackets separated by a colon.

Syntax: List [StartPosition : EndPosition : Step]

PARAMETERS	DESCRIPTION
StartPosition	Item at which we want to start. Default is the first character.
EndPosition	Up to which item we want to go (Note: This is up to the item at this specific location will not be included) Default is the last item.
Step	The number of steps we want it to take. By default, it is 1, means it will take one step at a time.

Fig: Lists Parameters

Example 1:

```
names = ['Steve', 'Sundar', 'Rahul', 'Sandeep', 'Vivek']
print (names [:2])
```

Output:

```
['Steve', 'Sundar']
```

Example 2:

```
print (names [1:])
print (names [1:3])
print (names [1:4:2])
```

Output:

```
['Sundar', 'Rahul', 'Sandeep', 'Vivek']
['Sundar', 'Rahul']
['Sunder', 'Sandeep']
```

➤ NEGATIVE INDEXING

We can also use Negative Indexing. The last item is

assigned with the index position of -1. It keeps on increasing in the negative order until the first item.

Example 1:

```
name = ['Steve', 'Sundar', 'Rahul', 'Sandeep', 'Vivek']
print (names [-2])
```

Output:

```
Sandeep
```

Example 2:

```
print (names[-2:-5])
print (names [-2:-5:-1])
print (names [::-1])
```

Output:

```
[]
['Sandeep', 'Rahul', 'Sundar']
['Vivek', 'Sandeep', 'Rahul', 'Sundar', 'Steve']
```

➤ INDEX OUT OF RANGE ERROR :-

If we try to access any index position where no item is present, then we get an 'index out of range' error.

Example:

```
names = ['Steve', 'Sundar', 'Rahul']
print (names [5])
```

Output:

```
IndexError: list index out of range.
```

➤ LOOPING A LIST :⇒

Using for loop

We can iterate over all the items in the list using a for loop. This is the most common way to iterate through a list.

Example:

```
birdslist = ["Parrot", "Crow", "Eagle"]
for bird in birdslist:
    print (bird)
```

Output:

```
Parrot
Crow
Eagle
```

➤ MEMBERSHIP TEST :⇒

Membership operators in and not in are used to check whether an item is present in the list.

Example:

```
name = ['Logical', 'Python']

if 'Logical' in name:
    print ("Logical")
```

Output:

```
Logical
```

➤ CHECK LENGTH OF A LIST :->

The len() function is used to check the length of a list.

Example:

```
my_list = [1, 2, 3, 4, 5]
print (len(my_list))
```

Output:

```
5
```